Sesión 1: Introducción al Procesamiento Paralelo

By Elias Cassal

Sesión 1: Introducción al Procesamiento Paralelo

- **Sesión 1: Introducción al Procesamiento Paralelo**
- ¡Hola a todos! Bienvenidos a nuestra primera sesión de Arquitectura de Computadores. Hoy vamos a explorar un concepto fundamental que define la computación moderna: el **procesamiento paralelo**. No se trata solo de que las computadoras sean más rápidas, sino de cómo están diseñadas para manejar la inmensa cantidad de información y tareas de nuestro mundo digital.

1. Introducción al Procesamiento Paralelo: Más Allá de la Velocidad

- Para entender por qué el paralelismo es tan vital, imaginemos un equipo de construcción. Si queremos construir una pared rápidamente, no le pedimos a una sola persona que trabaje el doble de rápido. En su lugar, ¡ponemos a varias personas a construir diferentes secciones de la pared al mismo tiempo! Esa es la esencia del procesamiento paralelo: **ejecutar múltiples instrucciones o tareas simultáneamente utilizando varios procesadores o núcleos.**

1. Introducción al Procesamiento Paralelo: Más Allá de la Velocidad

- Históricamente, los procesadores se hacían más rápidos simplemente aumentando su **frecuencia de reloj** (la velocidad a la que realizan operaciones). Pero esta estrategia chocó con límites físicos importantes:
- **Generación de Calor:** Mayor frecuencia = más calor. Los chips se sobrecalentaban, volviéndose inestables e inviables para seguir aumentando la velocidad.
- **Consumo Energético:** Mayor frecuencia = mayor consumo de energía. Esto es un problema tanto para dispositivos móviles como para grandes centros de datos.

1. Introducción al Procesamiento Paralelo: Más Allá de la Velocidad

- Aquí es donde el paralelismo nos da una solución elegante: en lugar de hacer que un solo procesador trabaje más rápido, hacemos que **múltiples procesadores trabajen en conjunto**. Esto nos permite obtener una potencia de cómputo mucho mayor sin enfrentar los problemas de sobrecalentamiento y consumo de energía que frenaron a los procesadores de un solo núcleo.

2. Motivación y Desafíos del Procesamiento Paralelo

- El procesamiento paralelo no es una opción; es la **columna vertebral de la computación moderna**. Sin él, muchas de las tecnologías que damos por sentadas simplemente no existirían o serían inviables.
- **¿Por qué es tan importante hoy?**
- **Grandes Volúmenes de Datos (Big Data):** ¿Se imaginan procesar terabytes de transacciones bancarias o datos de redes sociales con un solo procesador? Sería imposible en un tiempo útil. El paralelismo permite dividir y conquistar estos volúmenes masivos.
- **Simulaciones Complejas:** La predicción del tiempo, el diseño de vehículos o el modelado molecular requieren billones de cálculos por segundo. El paralelismo divide estas simulaciones en partes que se resuelven concurrentemente.
- **Inteligencia Artificial (IA) y Machine Learning (ML):** El entrenamiento de modelos de Deep Learning, con sus inmensas operaciones matriciales, solo es factible gracias al procesamiento paralelo masivo, especialmente en las **GPUs**.
- **Videojuegos y Realidad Virtual:** Para renderizar gráficos 3D complejos y simular físicas realistas en tiempo real, las consolas y computadoras usan procesadores multinúcleo y GPUs con miles de núcleos paralelos, creando experiencias inmersivas.
- **Servidores Web y Computación en la Nube:** Los centros de datos que alimentan Internet y la nube manejan millones de solicitudes simultáneas gracias a miles de servidores trabajando en paralelo.

Pero, ¿es todo color de rosa? No. El paralelismo también presenta sus propios desafíos:

- Overhead de Comunicación: Cuando los procesadores trabajan juntos, necesitan comunicarse. Esta coordinación introduce latencia y consume recursos. ¡No es gratis!
- Condiciones de Carrera y Sincronización: Si múltiples partes de un programa acceden y modifican los mismos datos al mismo tiempo, pueden surgir errores. Necesitamos mecanismos como locks o semáforos para coordinar estos accesos.
- Escalabilidad No Lineal: A menudo, duplicar el número de procesadores no significa duplicar la velocidad. Hay una parte del programa que siempre será secuencial, limitando el speedup. Esto lo veremos a fondo con la Ley de Amdahl.

3. Conceptos Clave: Concurrencia vs. Paralelismo

- Estos dos términos a menudo se usan indistintamente, pero son diferentes:
- **Concurrencia:** Trata con **múltiples tareas que pueden progresar de forma independiente**. Es la *capacidad* de un sistema para manejar múltiples tareas a la vez, no necesariamente *simultáneamente*.
  - **Ejemplo:** Un chef que tiene varias sartenes en el fuego y va alternando su atención entre ellas. Pareciera que cocina varias cosas a la vez, pero solo está en una a la vez. En una CPU, esto puede ser el *multitasking* o *multithreading* en un solo núcleo (cambio rápido de contexto).

3. Conceptos Clave: Concurrencia vs. Paralelismo

- **Paralelismo:** Implica la **ejecución *simultánea* de múltiples tareas o subpartes de una tarea**.
  - **Ejemplo:** Varios chefs trabajando en sus propias estaciones, cada uno cocinando un plato diferente al mismo tiempo. En una CPU, esto sucede cuando tienes múltiples núcleos o procesadores que realmente están ejecutando instrucciones en paralelo.
- **La relación:** El paralelismo es una forma de lograr concurrencia. Puedes tener concurrencia sin paralelismo (un solo núcleo simulando multitarea), pero no puedes tener paralelismo sin concurrencia (para que múltiples núcleos trabajen juntos, las tareas deben ser independientes o coordinables).

4. Taxonomía de Flynn: Clasificando las Arquitecturas Paralelas

- Para entender los diferentes tipos de arquitecturas paralelas, usamos la **Taxonomía de Flynn**. Esta clasificación se basa en cuántos flujos de instrucciones y flujos de datos puede manejar un sistema simultáneamente.
- **SISD (Single Instruction, Single Data)**:
  - **Una instrucción** a la vez, operando sobre **un solo flujo de datos** a la vez.
  - **Ejemplo:** La mayoría de las **CPUs tradicionales (procesadores de un solo núcleo)** que ejecutan programas secuenciales. Es el modelo clásico.
- **SIMD (Single Instruction, Multiple Data)**:
  - **Una instrucción** a la vez, operando sobre **múltiples flujos de datos** simultáneamente.
  - **Ejemplo:** Las **GPUs (Unidades de Procesamiento Gráfico)**. Cuando una GPU renderiza una imagen, aplica la misma instrucción (ej. "sumar luz a este píxel") a miles de píxeles (datos) al mismo tiempo. También se usa en procesadores vectoriales y extensiones de CPU (SSE, AVX).

4. Taxonomía de Flynn: Clasificando las Arquitecturas Paralelas

- **MISD (Multiple Instruction, Single Data)**:
  - **Múltiples instrucciones** ejecutándose simultáneamente sobre **un solo flujo de datos**.
  - **Ejemplo:** Es el tipo menos común. A veces se cita el **procesamiento de pipelines** o sistemas de control de vuelo tolerantes a fallos (varios procesadores ejecutan la misma instrucción y comparan resultados para asegurar fiabilidad).
- **MIMD (Multiple Instruction, Multiple Data)**:
  - **Múltiples instrucciones** ejecutándose simultáneamente sobre **múltiples flujos de datos**.
  - **Ejemplo:**
    - **Multiprocesadores (CPUs con múltiples núcleos):** Cada núcleo es una unidad de procesamiento independiente ejecutando sus propias instrucciones sobre sus propios datos.
    - **Clusters de computadoras:** Múltiples computadoras interconectadas, donde cada máquina es un nodo independiente que ejecuta su propio código sobre sus propios datos.
    - **Sistemas distribuidos:** Como los que alimentan la nube (servidores de Google, Facebook, Netflix).

Actividad en Clase: Discusión Guiada y Análisis de Arquitecturas en la Vida Real

- Ahora, con estos conceptos en mente, vamos a aterrizar esto en la realidad.
- **Instrucciones (Trabajo en Pequeños Grupos o Discusión Plenaria):**
- **Seleccionen una de las siguientes aplicaciones/sistemas:**
  - Un videojuego moderno (ej. FIFA, Call of Duty, Cyberpunk 2077)
  - Un servicio de *streaming* de video (ej. Netflix, YouTube)
  - Un software de edición de video o diseño gráfico (ej. Adobe Premiere, Photoshop)
  - Un sistema de bases de datos grande (ej. un banco, una red social)
  - Un sistema de predicción meteorológica o simulación científica
  - Su teléfono móvil (considerando su CPU y GPU)

Actividad en Clase: Discusión Guiada y Análisis de Arquitecturas en la Vida Real

- **Analicen su elección y respondan:**
  - **¿Dónde creen que esta aplicación utiliza el procesamiento paralelo?** Sean específicos (ej. "el motor de renderizado", "el procesamiento de datos del usuario", "el entrenamiento del modelo de IA").
  - **¿Qué tipo(s) de la Taxonomía de Flynn creen que están presentes en su arquitectura o en la forma en que procesa la información?** Justifiquen su respuesta. Por ejemplo, ¿ven una operación SIMD? ¿MIMD? ¿Ambas?
  - **¿Cuál sería el impacto si esta aplicación no usara procesamiento paralelo?** ¿Qué problemas surgirían? (Piensen en tiempo, memoria, experiencia de usuario).
  - **De los desafíos del paralelismo que discutimos (comunicación, sincronización, escalabilidad no lineal), ¿cuál creen que sería el más relevante para su aplicación elegida y por qué?**
- **Tiempo de la actividad:** 30-45 minutos para la discusión en grupos, 15-20 minutos para la puesta en común y debate general.

Sesion 2

- Discusion en plenaria por grupos…

Sesión de Clase 3:
1.2.2 Redes de Interconexión en Arquitecturas Paralelas

Redes de Interconexión en Arquitecturas Paralelas

- La forma en que los procesadores y la memoria se conectan entre sí es fundamental para el rendimiento de un sistema paralelo. La **red de interconexión** es el "camino" por donde viajan los datos y las instrucciones, y su diseño impacta directamente en la latencia, el ancho de banda y la escalabilidad del sistema.
- Las redes de interconexión se pueden clasificar según su topología (la disposición de sus nodos y enlaces), su forma de conmutación (cómo se enrutan los mensajes) y su control (cómo se gestiona el tráfico).

Objetivos de Aprendizaje:

- Al final de esta sesión, los estudiantes serán capaces de:
- **Explicar** la importancia de las redes de interconexión en el rendimiento y escalabilidad de sistemas paralelos.
- **Clasificar** y **describir** las principales topologías de red (directas e indirectas) con sus ventajas y desventajas.
- **Diferenciar** y **comparar** los modos de conmutación de circuitos, paquetes y wormhole, explicando sus escenarios de aplicación.
- **Analizar** las implicaciones del control de red (centralizado vs. distribuido) en la robustez y escalabilidad.
- **Relacionar** el diseño de la red de interconexión con el rendimiento general de una arquitectura paralela.

Introducción y Fundamentos

- **Pregunta de activación:**

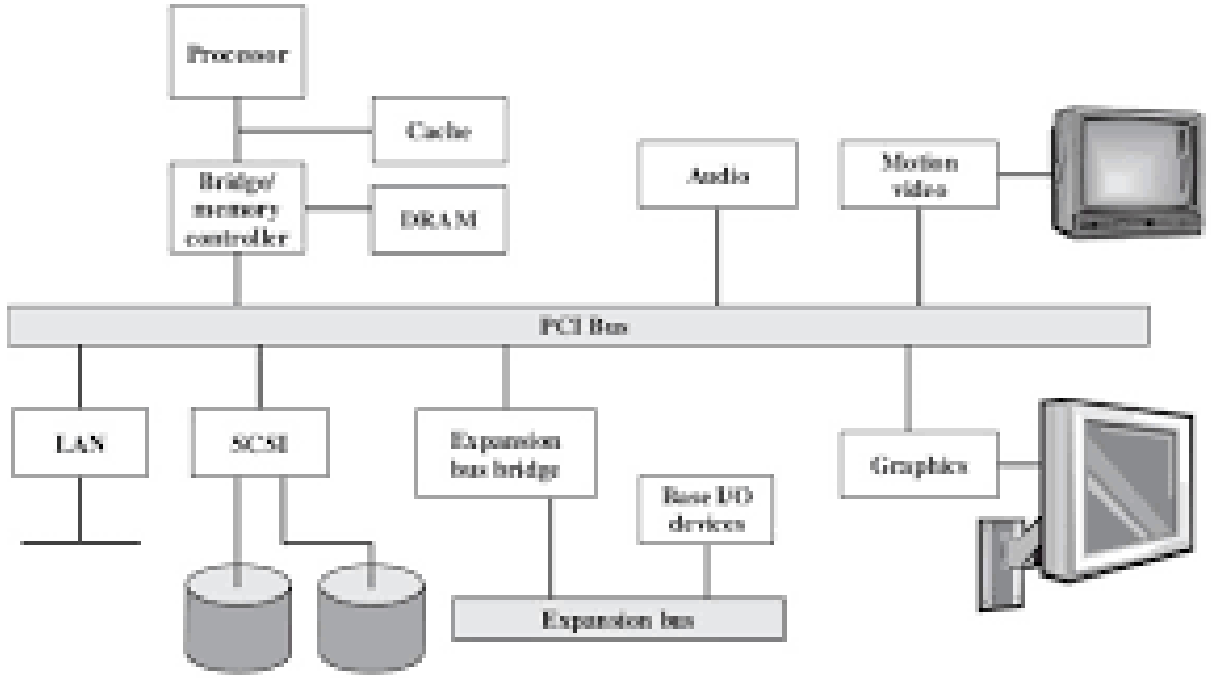
  "En la última sesión hablamos de los tipos de arquitecturas paralelas. ¿Qué recuerdan que hacía la memoria compartida? ¿Y los multicomputadores?" (Rápido feedback de los estudiantes).

- **Introducción al tema:**

  "Hoy nos adentraremos en el 'sistema nervioso' de estas arquitecturas: las **redes de interconexión**. Sin una forma eficiente de que los procesadores se 'hablen' entre sí o con la memoria, incluso el procesador más rápido se vuelve inútil en un sistema paralelo."

¿Qué es una Red de Interconexión y por qué es Fundamental?

- **Definición:** "La red de interconexión es el 'camino' por donde viajan los datos y las instrucciones entre procesadores, módulos de memoria, y periféricos en un sistema paralelo."



(a) Typical desktop system

Importancia (Analogía):

- "Imaginen una ciudad con muchas fábricas (procesadores) y almacenes (memoria). Si las calles (red de interconexión) son pequeñas y están congestionadas, no importa qué tan eficientes sean las fábricas o los almacenes, la producción se detendrá."
- "En computación paralela, la red impacta directamente en:
  - **Latencia:** ¿Cuánto tiempo tarda un mensaje en ir de A a B? (Tiempo de viaje en carretera).
  - **Ancho de Banda:** ¿Cuánta información puede viajar a la vez? (Capacidad de la carretera, número de carriles).
  - **Escalabilidad:** ¿Qué tan fácil es añadir más procesadores sin colapsar la red? (¿Podemos añadir más fábricas sin que las calles colapsen?).“

  Componentes conceptuales: Nodos (procesadores/memoria), Enlaces (conexiones físicas), Conmutadores (switches, para dirigir el tráfico).

Actividad: "Mapa Mental Rápido" (5 min)

- Individualmente o en parejas, escriban o dibujen rápidamente en una hoja de papel tres palabras clave o un pequeño esquema de lo que creen que podría hacer que una "carretera" (red) sea buena o mala para el tráfico de datos. (Ej: Muchos carriles, muchos semáforos, curvas cerradas, etc.).

compartir un par de ideas.

Clasificaciones de Redes de Interconexión

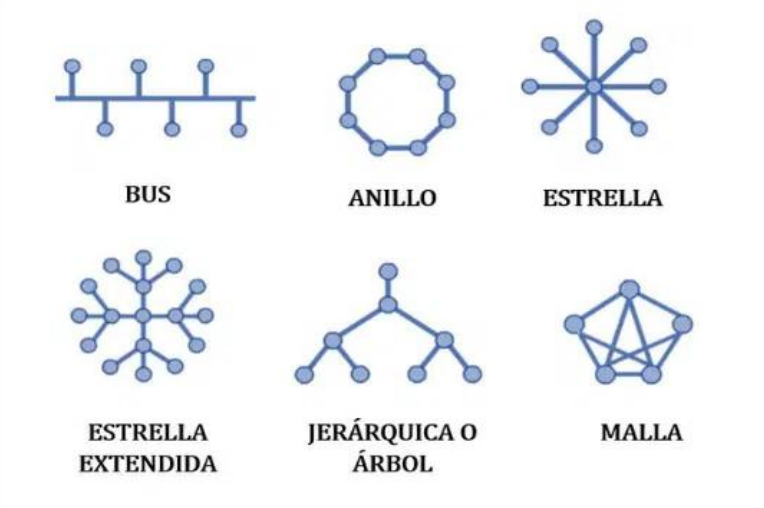
"Las redes de interconexión se pueden clasificar según tres aspectos principales que determinan su comportamiento:"

- **A. Topología de Red:** La disposición física o lógica de sus nodos y enlaces. ¿Cómo están conectados? (El mapa de carreteras de nuestra ciudad).
- **B. Modos de Conmutación:** Cómo los mensajes se enrutan a través de la red. ¿Cómo se decide qué carril toma un camión?
- **C. Control de la Red:** Quién toma las decisiones de enrutamiento y asignación de recursos. ¿Quién es el 'ingeniero de tráfico' de la ciudad?

"Vamos a explorar cada una de estas clasificaciones en detalle."

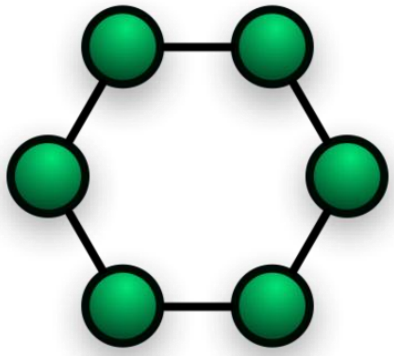
Parte 2: Topologías de Red (45 minutos)

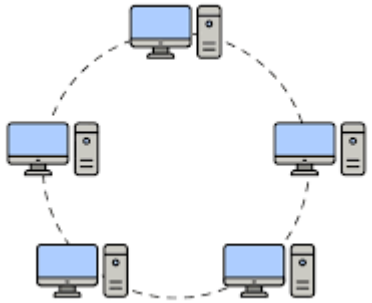
1. **Topologías Directas (20 min)**
    1. **Definición:** "Los nodos están conectados directamente entre sí, formando una estructura específica. No hay un switch central." (Pensar en redes entre vecinos o fábricas con rutas directas).
    2. **Descripción v Pros/Contras de cada una. con visualización:**



Anillo (Ring):

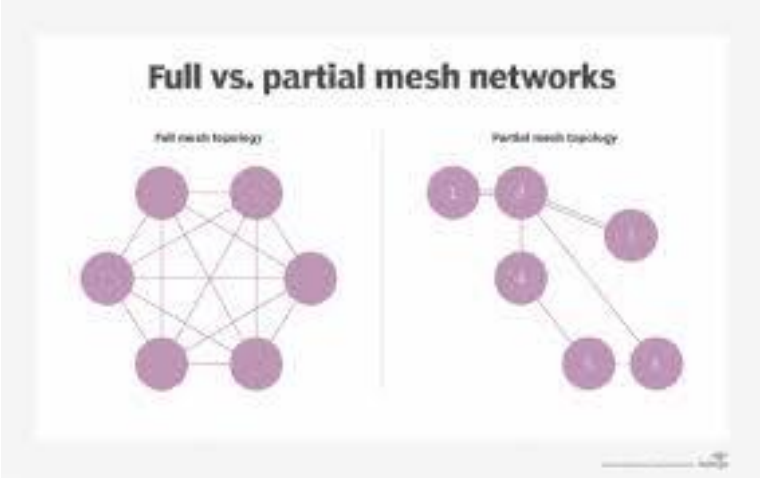
- **Descripción:** Nodos conectados en un círculo (P1-P2-P3-P1).
- **Visualización:** Dibujar en la pizarra. Pedir a 4 estudiantes que formen un círculo y se tomen de las manos. "Un mensaje de P1 a P3 debe pasar por P2."
- **Pros:** Simple, económico.
- **Contras:** Latencia alta para nodos distantes (muchos saltos). Baja tolerancia a fallos (un nodo/enlace roto colapsa el anillo).





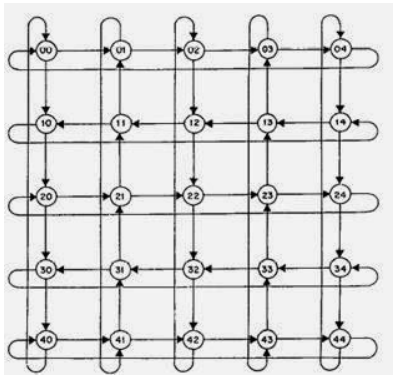
Malla (Mesh):

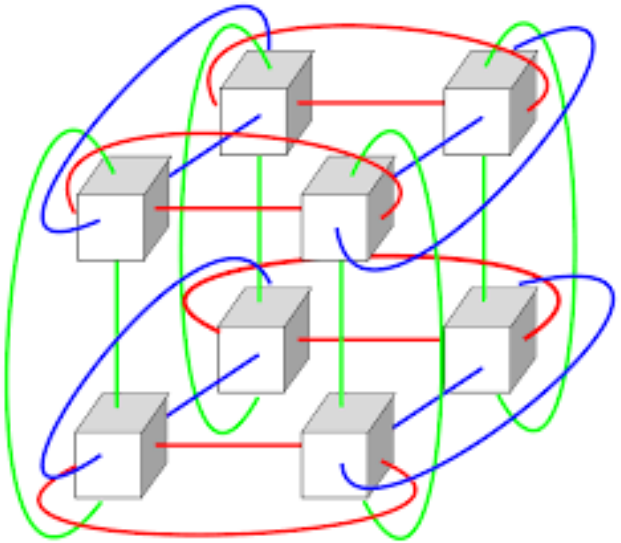
- **Descripción:** Nodos organizados en una cuadrícula 2D (o 3D). Cada nodo se conecta a sus vecinos adyacentes.
- **Visualización:** Dibujar una cuadrícula 3x3 en la pizarra. "P1,1 se conecta a P1,2 y P2,1."
- **Pros:** Permite múltiples rutas (redundancia), fácil escalabilidad por adición.
- **Contras:** Comunicación entre nodos muy separados puede requerir muchos saltos. Menos conectividad global que otras.



Toro (Torus):

- **Descripción:** Una malla con conexiones adicionales en los bordes (los nodos de los extremos se conectan a los nodos opuestos, como un donut).
- **Visualización:** Dibujar una malla y añadir las conexiones envolventes. "Imagina que el tablero de ajedrez se dobla para que los bordes opuestos se toquen."
- **Pros:** Reduce la distancia máxima entre nodos, mejora redundancia, mejor rendimiento que malla.
- **Contras:** Más compleja de cablear que la malla simple





Hipercubo (Hypercube):

- **Descripción:** Topología compleja donde el número de conexiones por nodo aumenta logarítmicamente con el número total de nodos (2^d nodos, cada nodo tiene d conexiones).
- **Visualización:** Dibujar un cubo para d=3. "Cada vértice es un procesador, cada arista es un enlace."
- **Pros:** Ofrece muchas rutas cortas (baja latencia promedio), alta conectividad.
- **Contras:** Costosa de construir a gran escala (muchos enlaces por nodo). Poca escalabilidad práctica más allá de ciertas dimensiones.

Árbol (Tree):

- **Descripción:** Nodos conectados de forma jerárquica.
- **Visualización:** Dibujar un árbol binario. "El nodo raíz es el 'coordinador', las hojas son los procesadores."
- **Pros:** Comunicación rápida si los datos fluyen hacia arriba o abajo en el árbol, simple.
- **Contras:** El nodo raíz (o nodos superiores) puede convertirse en un **cuello de botella** de ancho de banda o punto único de fallo.

Actividad Grupal: "Diseña tu Topología Ideal" (15 min)

- Divide a los estudiantes en grupos de 3-4.
- **Escenario:** "Están diseñando un supercomputador para el pronóstico del tiempo. Necesita alta velocidad para procesar datos de miles de sensores y realizar simulaciones masivas. ¿Qué topología directa elegirían y por qué? Dibújenla y justifiquen su elección basándose en latencia, ancho de banda y escalabilidad."
- Pide a cada grupo que comparta brevemente su elección y justificación. (Fomenta el debate y la aplicación de los conceptos).

Topologías Indirectas

(10 min)

- **Definición:** "Los nodos se conectan a través de un componente central o una red de conmutadores, no directamente entre sí." (Pensar en internet, donde los dispositivos se conectan a routers/switches, no directamente entre sí).

Descripción y Pros/Contras:

- **Bus:**
  - **Descripción:** Todos los procesadores comparten un único canal de comunicación.
  - **Visualización:** Dibujar un bus lineal con procesadores conectados.
  - **Pros:** Simple, económico.
  - **Contras:** El bus se convierte rápidamente en un **cuello de botella de ancho de banda** a medida que aumenta el número de procesadores. Muy común en sistemas multiprocesador de memoria compartida (SMP) de pocos núcleos. No escalable.

Descripción y Pros/Contras:

Conmutador Cruzado (Crossbar Switch):

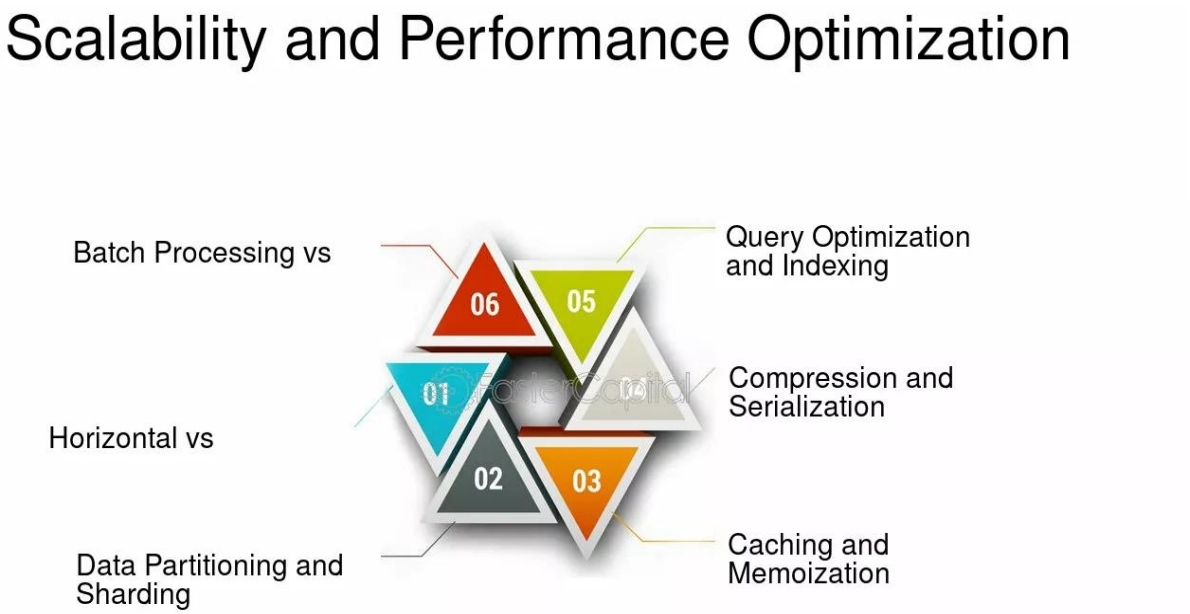
- **Descripción:** Permite que cualquier procesador se conecte directamente con cualquier módulo de memoria o con otro procesador sin conflictos, siempre que no intenten acceder al mismo destino simultáneamente.
- **Visualización:** Dibujar una cuadrícula de switches donde las líneas horizontales se cruzan con verticales, y en cada cruce hay un conmutador.
- **Pros:** Ofrece alto ancho de banda, baja latencia (conexiones directas).
- **Contras:** Su complejidad y costo aumentan **cuadráticamente** (N2) con el número de nodos, haciéndolo inviable para grandes sistemas.

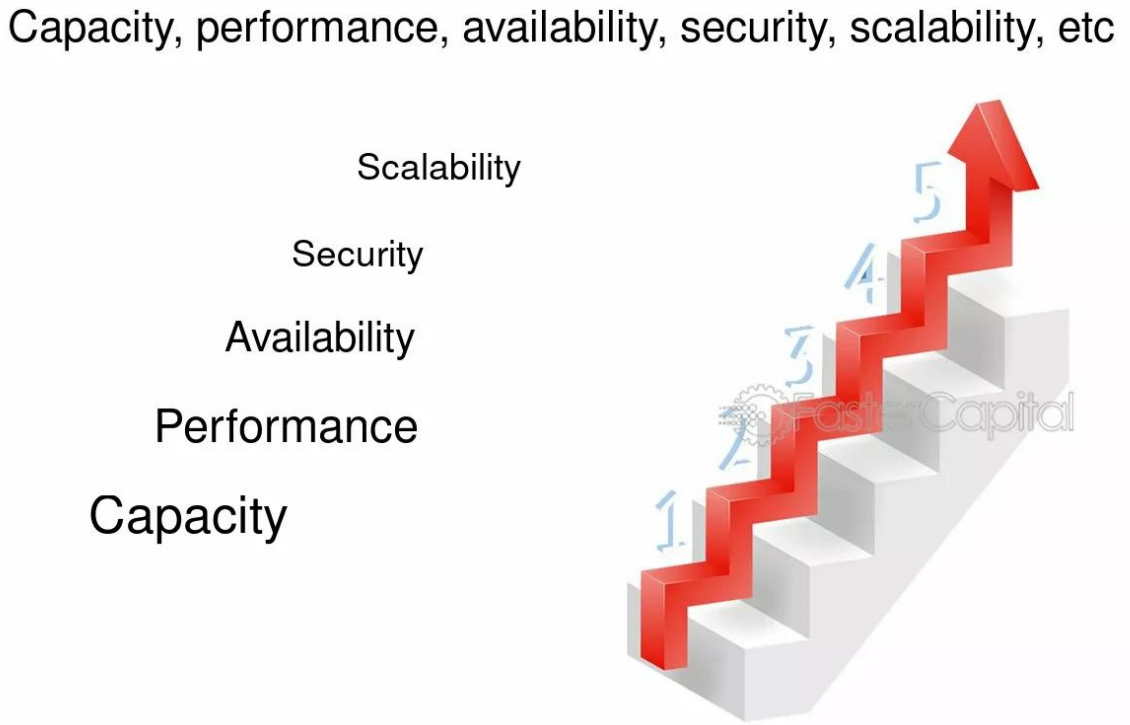
Descripción y Pros/Contras:

Red Multietapa (Multistage Interconnection Network - MIN):

- **Descripción**: Consiste en múltiples etapas de pequeños conmutadores interconectados. Son un compromiso.
- **Visualización**: Dibujar un ejemplo simple de red Omega o Butterfly (dos etapas con 2x2 switches).
- **Pros**: Compromiso entre bus (lento) y crossbar (caro), ofreciendo escalabilidad y buen rendimiento.
- **Contras**: Mayor latencia que el crossbar. Mayor complejidad que el bus. Posibilidad de bloqueos internos.

Reflexión: "¿Por qué creen que las MINs son tan importantes para los supercomputadores modernos?"





Parte 3: Modos de Conmutación y Control de la Red (35 minutos)

- **Modos de Conmutación (20 min)**
  - **Definición:** "Una vez que tenemos la topología, ¿cómo viaja la información a través de ella? Esto es el modo de conmutación."
  - **Conmutación de Circuitos:**
    - **Descripción:** Se establece una ruta física dedicada (un "circuito") entre origen y destino antes de que comience la transmisión de datos, similar a una llamada telefónica.
    - **Analogía:** "Es como reservar una autopista completa solo para tu camión durante todo el viaje."
    - **Pros:** Una vez establecido, el circuito es exclusivo, garantiza ancho de banda, baja latencia *una vez establecido*. Ideal para grandes transferencias de datos continuas.
    - **Contras:** Tiempo de establecimiento del circuito (handshake). Desperdicio de recursos si el circuito no se usa continuamente. Bloqueo de rutas.

Conmutación de Paquetes:

- **Descripción:** Los datos se dividen en pequeños "paquetes" independientes que se enrutan individualmente a través de la red. Cada paquete puede tomar una ruta diferente. Cada nodo almacena y reenvía el paquete completo.
- **Analogía:** "Es como enviar muchas cartas por correo. Cada carta va por su lado, puede tomar rutas diferentes, y llega independientemente."
- **Pros:** Más flexible y eficiente para tráfico irregular (Internet). Uso eficiente del ancho de banda compartido. Tolerancia a fallos (rutas alternativas).
- **Cons:** Latencia variable (colocación en cola en cada nodo), requiere búferes en cada nodo, sobrecarga de cabeceras de paquetes.

Conmutación de Wormhole (Gusano):

- **Descripción:** Una optimización de la conmutación de paquetes donde los datos fluyen a través de la red **tan pronto como llega la cabecera del paquete**, sin esperar a que el paquete completo esté disponible en cada nodo. El paquete se divide en "flits" (flow control units).
- **Analogía:** "El gusano es la cabecera del paquete, que 'perfora' un camino a través de los nodos. El resto del cuerpo (los datos) lo sigue de cerca. Si el gusano choca con un obstáculo, todo el cuerpo se detiene hasta que el camino se despeja, pero no se almacena en cada nodo completo."
- **Pros:** Reduce drásticamente la latencia (no se espera a que el paquete completo llegue al nodo intermedio). Reduce el tamaño de los búferes necesarios en cada conmutador. Muy común en supercomputadoras y redes *on-chip*.
- **Contras:** Si la cabecera se bloquea, toda la ruta está ocupada hasta que se libera. Puede llevar a *deadlocks* (interbloqueos) si no se gestiona bien el enrutamiento.

Control de la Red (10 min)

- **Centralizado:**
  - **Descripción:** Un único controlador (un nodo o un hardware específico) gestiona todas las decisiones de enrutamiento y asignación de recursos para toda la red.
  - **Analogía:** "Un solo ingeniero de tráfico que ve toda la ciudad y decide dónde va cada camión."
  - **Pros:** Simple de implementar y depurar. Fácil para optimizar el tráfico globalmente.
  - **Contras: Punto único de fallo**. **Cuello de botella de escalabilidad** (el controlador se satura con muchas peticiones). Latencia de comunicación con el controlador.

Distribuido

- :**Descripción**: Las decisiones de enrutamiento y control se toman en cada nodo de forma independiente, basándose en la información local y en protocolos compartidos entre nodos.
- **Analogía**: "Cada conductor de camión (o cada intersección) tiene reglas simples que les permiten decidir la mejor ruta en su zona, comunicándose con otros conductores.“
- **Pros**: Más robusto (sin punto único de fallo). Mucho más escalable (la carga se distribuye). Menor latencia de decisión.
- **Contras**: Más complejo de diseñar e implementar. Puede ser más difícil optimizar el tráfico globalmente. Riesgo de deadlocks o livelocks si las reglas no son perfectas.

Pregunta de Reflexión: "¿Por qué creen que los sistemas paralelos a gran escala casi siempre usan control distribuido?"





Parte 4: Resumen y Cierre (15 minutos)

1. **Preguntas y Respuestas (10 min)**
    1. Abrir el foro para cualquier duda.
    2. **Preguntas dirigidas para reforzar:**
        1. "¿Pueden dar un ejemplo de una aplicación que se beneficiaría más de una topología de Malla que de un Anillo?"
        2. "¿Por qué el Crossbar no es práctico para un supercomputador de 1000 procesadores, pero el Bus tampoco?"
        3. "Si están diseñando un sistema donde la latencia es **absolutamente crítica** y las transferencias de datos son **grandes y continuas**, ¿qué modo de conmutación sería su primera opción? ¿Y por qué el Wormhole es mejor que el de Paquetes para alto rendimiento?"
        4. "¿Qué problema fundamental del Bus intenta resolver una Red Multietapa?"

Conclusión(5 min)

- "Hoy hemos visto que la red de interconexión es el sistema circulatorio de las arquitecturas paralelas. Su diseño es tan crítico como los propios procesadores y la memoria."
- "La elección de una **topología** (cómo se conectan), un **modo de conmutación** (cómo viajan los datos) y un **esquema de control** (quién gestiona el tráfico) impacta directamente en el rendimiento, el costo, la escalabilidad y la fiabilidad de cualquier sistema paralelo."
- "Comprender estas clasificaciones es fundamental para poder diseñar o seleccionar la arquitectura paralela adecuada para un problema computacional dado. No existe una solución única para todo; la mejor red es la que se adapta a las necesidades específicas de la aplicación."
- "En las próximas sesiones, exploraremos cómo estas redes se implementan en arquitecturas específicas y cómo el software interactúa con ellas."